

Reinforcement Learning for Taxi-v3 Environment: A Comparative Study of Q-Learning and Deep Q-Networks

Igor Pasquini, 2024218

November 4, 2025

Abstract

This report presents a comprehensive study of reinforcement learning approaches applied to the Taxi-v3 environment from OpenAI Gymnasium. We implement and compare two fundamental RL algorithms: tabular Q-learning and Deep Q-Networks (DQN). Both methods are evaluated under deterministic and stochastic environment conditions. Our implementation demonstrates successful convergence for both approaches, with detailed analysis of their architectural differences, learning dynamics, and performance characteristics.

Contents

1	Problem Description	3
1.1	The Taxi-v3 Environment	3
1.2	Environment Variants	3
2	Solution Approaches	4
2.1	Tabular Q-Learning	4
2.1.1	Theoretical Background	4
2.1.2	Exploration Strategy	4
2.1.3	Algorithmic Procedure	4
2.1.4	Advantages and Limitations	5
2.2	Deep Q-Network (DQN)	5
2.2.1	Motivation	5
2.2.2	Architecture and Data Flow	5
2.2.3	Training Mechanism	6
2.2.4	Experience Replay	6
2.2.5	Optimization Process	6
2.2.6	Key Differences from Tabular Q-Learning	7
3	Implementation Architecture	8
3.1	Q-Learning Implementation	9
3.2	Deep Q-Network (DQN) Implementation	11
3.2.1	Neural Network Model	11
3.2.2	Replay Buffer	11
3.2.3	Agent Structure and Learning Routine	12
3.3	Performance Monitoring and Visualization	14
4	Experimental Results	15
4.1	Tabular Q-Learning	15
4.1.1	Experimental Setup	15
4.1.2	Deterministic Environment	16
4.1.3	Stochastic Environment	20
4.2	Deep Q-Network (DQN)	24
4.2.1	Experimental Setup	24
4.2.2	Deterministic Environment	24
4.2.3	Stochastic Environment	28
4.3	Comparative Analysis: Q-Learning vs DQN	30

1 Problem Description

1.1 The Taxi-v3 Environment

The Taxi-v3 environment is a classic reinforcement learning problem that simulates a taxi navigation task within a 5×5 grid world. The problem involves a discrete state space totaling 500 states, which is composed of the taxi's 25 possible grid positions, the passenger's location (four designated pickup spots plus one in-taxi state), and the four possible destination locations. The agent can choose from discrete actions: Move South, North, East, West, Pickup passenger, or Drop-off passenger. The primary objective is for the agent to pick up a passenger from a designated location, navigate to the passenger's destination, and successfully drop off the passenger at the correct location. The reward structure is designed to encourage efficiency, providing -1 for each time step taken, +20 for a successful drop-off, and -10 for any illegal pickup or drop-off attempts.

1.2 Environment Variants

We investigate two variants of this problem. The Deterministic Variant is a simpler learning problem in which actions always execute as intended, resulting in predictable state transitions. In contrast, the Stochastic Variant is significantly more challenging due to environmental uncertainty. This variant introduces two complexities:

- `is_rainy = bool`: If true the cab will move in intended direction with probability of 80% else will move in either left or right of target direction with equal probability of 10
- `fickle_passenger = bool`: If true the passenger has a 30% chance of changing destinations when the cab has moved one square away from the passenger's source location. Passenger fickleness only happens on the first pickup and successful movement. If the passenger is dropped off at the source location and picked up again, it is not triggered again.

2 Solution Approaches

This section describes in depth the two reinforcement learning techniques applied to the Taxi-v3 task: the **Tabular Q-Learning** algorithm and its neural counterpart, the **Deep Q-Network (DQN)**. Both methods aim to learn an optimal policy $\pi^*(s)$ that maximizes the expected cumulative reward, but they differ in how the function $Q(s, a)$ is represented and updated.

2.1 Tabular Q-Learning

2.1.1 Theoretical Background

Q-Learning is a model-free, off-policy reinforcement learning algorithm which operates by iteratively refining a value function $Q(s, a)$ that estimates the expected future return obtained by taking action a in state s and following the optimal policy thereafter. At each step, the agent updates its Q-value estimate according to the **Bellman Optimality Equation**:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (1)$$

Here:

- $\alpha \in (0, 1]$ is the learning rate, controlling how strongly new experiences override old estimates.
- $\gamma \in [0, 1)$ is the discount factor that determines how much future rewards influence the present.
- r is the immediate reward, and s' the next observed state.

Intuitively, the algorithm adjusts the current Q-value toward a target value composed of the observed reward plus the discounted value of the best next action. The difference between target and prediction, called the *temporal difference error*, quantifies how much the estimate needs to change.

2.1.2 Exploration Strategy

Q-learning uses an ϵ -greedy policy to balance exploration and exploitation. At each step, with probability ϵ the agent selects a random action (exploration), and with probability $1 - \epsilon$ it selects the action with the highest Q-value (exploitation). The exploration rate decays linearly during training:

$$\epsilon_t = \max(\epsilon_{\min}, \epsilon_0 - \text{decay_rate} \times t) \quad (2)$$

This schedule ensures that the agent explores extensively early on and gradually exploits more as it learns reliable value estimates.

2.1.3 Algorithmic Procedure

Algorithm 1 shows the canonical Q-learning process.

Algorithm 1 Tabular Q-Learning

```
1: Initialize Q-table  $Q(s, a) \leftarrow 0$  for all  $(s, a)$ 
2: for each episode do
3:   Reset environment and observe initial state  $s$ 
4:   while episode not terminated do
5:     Choose  $a$  from  $\epsilon$ -greedy policy using  $Q(s, a)$ 
6:     Execute  $a$ , observe reward  $r$  and next state  $s'$ 
7:     Update  $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ 
8:      $s \leftarrow s'$ 
9:   end while
10:  Decay  $\epsilon$ 
11: end for
```

2.1.4 Advantages and Limitations

The tabular approach is simple, interpretable, and provably convergent for finite Markov decision processes if all state-action pairs are explored sufficiently often. However, its main drawback is scalability: the Q-table grows linearly with the product of state and action spaces. For larger or continuous domains, this representation becomes infeasible. This motivates the transition to function approximation methods such as DQN.

2.2 Deep Q-Network (DQN)

2.2.1 Motivation

The Deep Q-Network (DQN) extends Q-learning by using a neural network to approximate the Q-function:

$$Q(s, a; \theta) \approx Q^*(s, a) \quad (3)$$

where θ are the network parameters (weights). Instead of storing a lookup table, the network generalizes across similar states, enabling learning in high-dimensional or continuous environments.

2.2.2 Architecture and Data Flow

In this implementation, the network receives a one-hot encoded state vector of size 500 (since the Taxi-v3 environment has 500 discrete states). It outputs six Q-values corresponding to the six possible actions. The architecture includes two fully connected hidden layers with ReLU activations.

Input (500) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Output(6)

The highest-valued output neuron corresponds to the action with the highest predicted future return.

2.2.3 Training Mechanism

The network is trained via gradient descent to minimize the difference between the predicted Q-value and a target computed using the Bellman equation. Given a transition $(s, a, r, s', \text{done})$, the target is defined as:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-) \quad (4)$$

where θ^- denotes the parameters of the target network. In this simplified version, the same network is used for both prediction and target computation, but the learning mechanism remains consistent with standard DQN logic.

The loss function is the **Smooth L1 loss** (similar to Huber loss):

$$L(\theta) = \mathbb{E}_{(s,a,r,s')} [\text{Huber}(y - Q(s, a; \theta))] \quad (5)$$

which behaves quadratically for small errors and linearly for large ones, improving robustness to outliers.

2.2.4 Experience Replay

To prevent strong correlations between consecutive updates, DQN employs an *experience replay buffer*. This structure stores recent transitions and allows the agent to sample random mini-batches during training, effectively breaking temporal dependencies and improving data efficiency.

Listing 1: Sampling mini-batches from the replay buffer.

```
1 batch = random.sample(self.buffer, batch_size)
2 states, actions, rewards, next_states, dones = zip(*batch)
```

2.2.5 Optimization Process

The network parameters are optimized with the AdamW optimizer and gradient clipping to avoid numerical divergence. The training loop involves:

1. Collecting transitions from environment interactions.
2. Storing them into the replay buffer.
3. Sampling random mini-batches and performing backpropagation on the loss.

Algorithm 2 Deep Q-Network Training

```
1: Initialize replay buffer  $\mathcal{D}$ , network  $Q(s, a; \theta)$ 
2: for each episode do
3:   Observe initial state  $s$ 
4:   while not terminated do
5:     Choose action  $a$  from  $\epsilon$ -greedy policy w.r.t.  $Q(s, a; \theta)$ 
6:     Execute  $a$ , observe reward  $r$ , next state  $s'$ , and termination flag done
7:     Store transition  $(s, a, r, s', \text{done})$  in  $\mathcal{D}$ 
8:     if  $|\mathcal{D}| > \text{batch size}$  then
9:       Sample random mini-batch  $\{(s_i, a_i, r_i, s'_i, \text{done}_i)\}$  from  $\mathcal{D}$ 
10:      Compute predicted Q-values:  $\hat{Q}_i = Q(s_i, a_i; \theta)$ 
11:      Compute target Q-values:
```

$$y_i = r_i + (1 - \text{done}_i) \gamma \max_{a'} Q(s'_i, a'; \theta)$$

```
12:      Compute loss:
```

$$L = \frac{1}{N} \sum_i (y_i - \hat{Q}_i)^2$$

```
13:      Perform a gradient descent step on  $L$  w.r.t.  $\theta$ 
14:    end if
15:     $s \leftarrow s'$ 
16:  end while
17:  Decay  $\epsilon$ 
18: end for
```

2.2.6 Key Differences from Tabular Q-Learning

- **Representation:** Q-Learning uses explicit tables, while DQN generalizes through a neural network.
- **Learning:** Tabular updates are immediate; DQN updates rely on stochastic gradient descent over replayed experiences.
- **Stability:** DQN uses replay and gradient clipping to stabilize training, compensating for the moving-target problem introduced by function approximation.

3 Implementation Architecture

This section presents the internal design and code organization of the two reinforcement learning agents developed for the Taxi-v3 environment: the **Q-learning** agent and the **Deep Q-Network (DQN)** agent. Both implementations were written in Python and structured to emphasize clarity, modularity, and reproducibility.

Each implementation follows a modular design pattern consisting of:

1. An *agent class* responsible for policy decisions and learning updates.
2. A *training loop* that manages environment interaction and exploration decay.
3. Auxiliary *evaluation and visualization* routines for testing and diagnostics.

3.1 Q-Learning Implementation

The Q-Learning approach uses a tabular representation of the state-action space through a Python defaultdict. The main logic is encapsulated in the TaxiAgent class, which manages exploration, learning updates, and epsilon decay.

Listing 2: Core structure of the TaxiAgent class.

```
1 class TaxiAgent:
2     def __init__(self, env, learning_rate, initial_epsilon,
3                 epsilon_decay, final_epsilon, discount_factor=0.95):
4         self.env = env
5         self.q_values = defaultdict(lambda: np.zeros(env.action_space.n))
6         self.lr = learning_rate
7         self.discount_factor = discount_factor
8         self.epsilon = initial_epsilon
9         self.epsilon_decay = epsilon_decay
10        self.final_epsilon = final_epsilon
11        self.training_error = []
```

The Q-table is automatically initialized with zeros for unseen states. During training, the agent chooses actions via an ϵ -greedy strategy: it selects a random action with probability ϵ , or the best-known action according to the Q-table otherwise.

Listing 3: Epsilon-greedy action selection.

```
1 def get_action(self, obs):
2     if np.random.random() < self.epsilon:
3         return self.env.action_space.sample() # Exploration
4     else:
5         return int(np.argmax(self.q_values[obs])) # Exploitation
```

After each step, the agent performs a temporal difference (TD) update using the Bellman equation. The TD error represents the discrepancy between predicted and observed returns, providing the correction signal for learning.

Listing 4: Temporal difference update rule.

```
1 def update(self, obs, action, reward, terminated, next_obs):
2     current_q = self.q_values[obs][action]
3     future_q = (not terminated) * np.max(self.q_values[next_obs])
4     target = reward + self.discount_factor * future_q
5     td_error = target - current_q
6     self.q_values[obs][action] += self.lr * td_error
7     self.training_error.append(td_error)
```

Epsilon decay is applied after every episode to gradually reduce exploration:

```
1 def decay_epsilon(self):
```

```
2 |         self.epsilon = max(self.final_epsilon, self.epsilon - self.  
    |             epsilon_decay)
```

Training Loop. The training process iterates through multiple episodes, allowing the agent to interact with the environment, update Q-values, and progressively shift from exploration to exploitation.

Listing 5: Simplified training loop for Q-Learning.

```
1 | for episode in range(n_episodes):  
2 |     obs, info = env.reset()  
3 |     done = False  
4 |     while not done:  
5 |         action = agent.get_action(obs)  
6 |         next_obs, reward, terminated, truncated, info = env.step(action  
    |             )  
7 |         agent.update(obs, action, reward, terminated, next_obs)  
8 |         obs = next_obs  
9 |         done = terminated or truncated  
10 |    agent.decay_epsilon()
```

The code also integrates `RecordEpisodeStatistics` from `Gymnasium` to collect episode rewards and lengths. The resulting arrays are used in the `plot_results()` function, which visualizes learning curves such as average reward, episode length, and TD error over time.

Finally, the `test_agent()` function evaluates the learned policy in pure exploitation mode ($\epsilon = 0$) to compute average rewards and success rates over thousands of independent test episodes.

3.2 Deep Q-Network (DQN) Implementation

The DQN architecture replaces the Q-table with a neural network that approximates $Q(s, a)$ values directly from state representations. The implementation is split into three main components:

1. The DQN neural network model.
2. The ReplayBuffer class for experience replay.
3. The DQNAgent class managing interaction, training, and exploration.

3.2.1 Neural Network Model

The network, implemented with PyTorch, consists of two hidden layers with ReLU activations and an output layer that predicts Q-values for each of the six possible actions.

Listing 6: DQN neural network definition.

```
1 class DQN(nn.Module):
2     def __init__(self, n_observations, n_actions):
3         super(DQN, self).__init__()
4
5         self.layer1 = nn.Linear(n_observations, 64)
6         self.layer2 = nn.Linear(64, 64)
7         self.layer3 = nn.Linear(64, n_actions)
8
9     def forward(self, x):
10        x = torch.relu(self.layer1(x))
11        x = torch.relu(self.layer2(x))
12        return self.layer3(x)
```

Each state is one-hot encoded before being passed into the network. The output vector contains one Q-value per action, from which the agent selects the maximum for exploitation.

3.2.2 Replay Buffer

To stabilize learning and remove temporal correlations, the agent stores past transitions $(s, a, r, s', done)$ in a circular buffer. Random minibatches are sampled for training.

Listing 7: Replay buffer structure.

```
1 class ReplayBuffer:
2     def __init__(self, capacity):
3         self.memory = deque(maxlen=capacity)
4
5     def push(self, state, action, next_state, reward, terminated):
6         self.memory.append((state, action, next_state, reward,
7                             terminated))
8
9     def sample(self, batch_size):
```

```

7         return random.sample(self.memory, batch_size)
8     def __len__(self):
9         return len(self.memory)

```

3.2.3 Agent Structure and Learning Routine

The DQNAgent class orchestrates exploration, experience storage, and neural updates. It maintains a PyTorch optimizer (AdamW with AMSGrad) and implements gradient clipping to prevent numerical instabilities.

Listing 8: DQNAgent class overview.

```

1 class DQNAgent:
2     def __init__(self, env, learning_rate, gamma, initial_epsilon,
3         epsilon_decay, final_epsilon, discount_factor, batch_size):
4         self.env = env
5
6         self.n_actions = env.action_space.n
7         self.n_observations = env.observation_space.n
8
9         self.lr = learning_rate
10        self.gamma = discount_factor
11        self.epsilon = initial_epsilon
12        self.epsilon_decay = epsilon_decay
13        self.final_epsilon = final_epsilon
14        self.batch_size = batch_size
15
16        self.policy_net = DQN(500, 6).to(device)
17        self.optimizer = torch.optim.AdamW(
18            self.policy_net.parameters(), lr=self.lr, amsgrad=True
19        )
20        self.replay_buffer = ReplayBuffer(10000)
21        self.criterion = nn.SmoothL1Loss()

```

The agent chooses actions using an ϵ -greedy strategy similar to the tabular case, but based on neural predictions:

Listing 9: DQN action selection.

```

1 def get_action(self, state):
2     if random.random() < self.epsilon:
3         return self.env.action_space.sample()
4
5     else:
6         state_tensor = torch.zeros(self.n_observations, device=device)
7         state_tensor[state] = 1.0
8         q_values = self.policy_net(state_tensor.unsqueeze(0))
9         return q_values.max(1)[1].item()

```

During training, random batches are sampled from the replay buffer to compute the Smooth L1 loss between predicted and target Q-values:

Listing 10: Training update for DQN.

```
1 def learn(self):
2     if len(self.buffer) < 64: # wait for sufficient samples
3         return
4     transitions = self.replay_buffer.sample(self.batch_size)
5     batch = Transition(*zip(*transitions))
6
7     state_batch = torch.zeros(self.batch_size, self.n_observations,
8                               device=device)
9     next_state_batch = torch.zeros(self.batch_size, self.n_observations
10                                   , device=device)
11
12     non_final_mask = torch.tensor(tuple(map(lambda s: s is not None,
13                                             batch.next_state)), device=device, dtype=torch.bool)
14
15     for i, s in enumerate(batch.state):
16         state_batch[i, s] = 1.0
17
18     next_state_indices = [s for s in batch.next_state if s is not None]
19     if len(next_state_indices) > 0:
20         next_state_batch[non_final_mask] = torch.eye(self.
21                                                         n_observations, device=device)[next_state_indices]
22
23     action_batch = torch.tensor(batch.action, device=device).unsqueeze(
24         1)
25     reward_batch = torch.tensor(batch.reward, device=device)
26
27     current_q_values = self.policy_net(state_batch).gather(1,
28                                                         action_batch)
29     next_q_values = torch.zeros(self.batch_size, device=device)
30
31     with torch.no_grad():
32         next_q_values[non_final_mask] = self.policy_net(
33             next_state_batch[non_final_mask]).max(1)[0]
34
35     targets = (next_q_values * self.gamma) + reward_batch
36
37     loss = self.criterion(current_q_values, targets.unsqueeze(1))
38     self.optimizer.zero_grad()
39     loss.backward()
40     torch.nn.utils.clip_grad_value_(self.policy_net.parameters(), 100)
41     self.optimizer.step()
```

The DQN agent's training loop mirrors the Q-learning loop but incorporates replay buffer updates and periodic learning calls. The epsilon value decays linearly after each episode to transition from exploration to exploitation.

Listing 11: Main DQN training loop.

```
1 for episode in range(num_episodes):
2     state, _ = env.reset()
3     done = False
```

```
4 while not done:
5     action = agent.get_action(state)
6     next_state, reward, terminated, truncated, _ = env.step(action)
7     agent.replay_buffer.push(state, action, next_state, reward,
8                             terminated)
9     agent.learn()
10    state = next_state
11    done = terminated or truncated
agent.decay_epsilon()
```

3.3 Performance Monitoring and Visualization

Both implementations track learning dynamics through:

- **Episode rewards** – cumulative reward per episode.
- **Episode length** – number of actions required to complete each task.
- **Training error / loss** – TD error for Q-learning, Smooth L1 loss for DQN.

The `plot_results()` function computes moving averages to smooth noisy learning curves.

4 Experimental Results

This chapter presents the results obtained from the training and evaluation of both the Tabular Q-Learning and the Deep Q-Network (DQN) approaches applied to the Taxi-v3 environment. Each method was trained under varying conditions and hyperparameters, in both deterministic and stochastic variants of the environment, to analyze learning stability and performance robustness.

4.1 Tabular Q-Learning

4.1.1 Experimental Setup

The Tabular Q-Learning algorithm represents the simplest reinforcement learning baseline for the Taxi-v3 environment. Unlike Deep Q-Learning, it does not approximate the value function with a neural network, but directly stores an explicit table of Q-values $Q(s, a)$ for each discrete state-action pair. Given that the Taxi-v3 environment has only 500 states and 6 actions, this tabular approach remains computationally feasible while providing full control over the learning dynamics.

Experiments were performed under two different environment variants:

- **Deterministic:** the default Taxi-v3 environment, where state transitions and rewards are predictable.
- **Stochastic:** a modified version including uncertain dynamics (`is_rainy=True`, `fickle_passenger=True`), introducing variability in passenger behavior and movement results.

Across both variants, several hyperparameters were tuned to observe their effect on learning performance:

- **Learning rate** α — controls how much new information overrides old estimates.
- **Discount factor** γ — determines how far future rewards are considered.
- **Exploration schedule** (ϵ_{start} , ϵ_{final} , ϵ_{decay}) — affects how quickly the agent shifts from exploration to exploitation.
- **Training episodes** — controls the total experience accumulated by the agent.

After training, each configuration was tested for 10 000 episodes with exploration disabled ($\epsilon = 0$). Reported metrics include:

- **Win Rate** — percentage of successful passenger deliveries.
- **Average Reward and Standard Deviation** — summarize the expected performance and consistency.

4.1.2 Deterministic Environment

The deterministic environment allows Q-learning to converge quickly toward the optimal policy, since transitions are fully predictable and each state-action pair can be accurately estimated through repeated experience.

Influence of Hyperparameters

- **Learning rate (α):** This parameter had the greatest influence on convergence speed. A small $\alpha = 0.005$ resulted in very slow updates, reaching only a 9.7% win rate after 10 000 episodes and 40.9% after 20 000. Increasing α to 0.01 accelerated convergence significantly, producing a 45.1% win rate at 10 000 and 92.5% at 20 000 episodes.
- **Epsilon decay (ϵ_{decay}):** Controls how quickly exploration is reduced. In these deterministic runs, I decided to use a moderate value, obtained as $\text{start_epsilon} / (\text{n_episodes} / 2)$, calculated to make epsilon reach 0 after 50% of the total episodes have passed. This is suitable for a deterministic environment.
- **Discount factor (γ):** A lower $\gamma = 0.9$ slightly decreased long-term reward accumulation (86.2% win rate) compared to $\gamma = 0.95$ (92.5%), but both eventually converged with extended training.
- **Episodes:** Increasing training duration improved performance monotonically, with full convergence reached by 50 000 episodes for $\alpha = 0.01$.

Short training: 10 000–20 000 episodes During early training, the agent’s performance was highly sensitive to both α and γ , with slower values performing worse with few episodes. It was clear that 10000 or 20000 episodes weren’t enough, so i fastly moved to tests with more episodes.

Table 1: Q-Learning results (Deterministic, short training).

Episodes	α	γ	Final ϵ	ϵ_{decay}	Win Rate (%)	Avg. Reward $\pm$ Std.
10 000	0.01	0.95	0.1	$\text{start_epsilon} / (\text{n_episodes} / 2)$	45.1	-105.38 ± 104.41
10 000	0.005	0.95	0.1	$\text{start_epsilon} / (\text{n_episodes} / 2)$	9.7	-179.44 ± 62.82
20 000	0.01	0.95	0.1	$\text{start_epsilon} / (\text{n_episodes} / 2)$	92.5	-7.48 ± 55.03
20 000	0.01	0.9	0.1	$\text{start_epsilon} / (\text{n_episodes} / 2)$	86.2	-20.37 ± 71.91

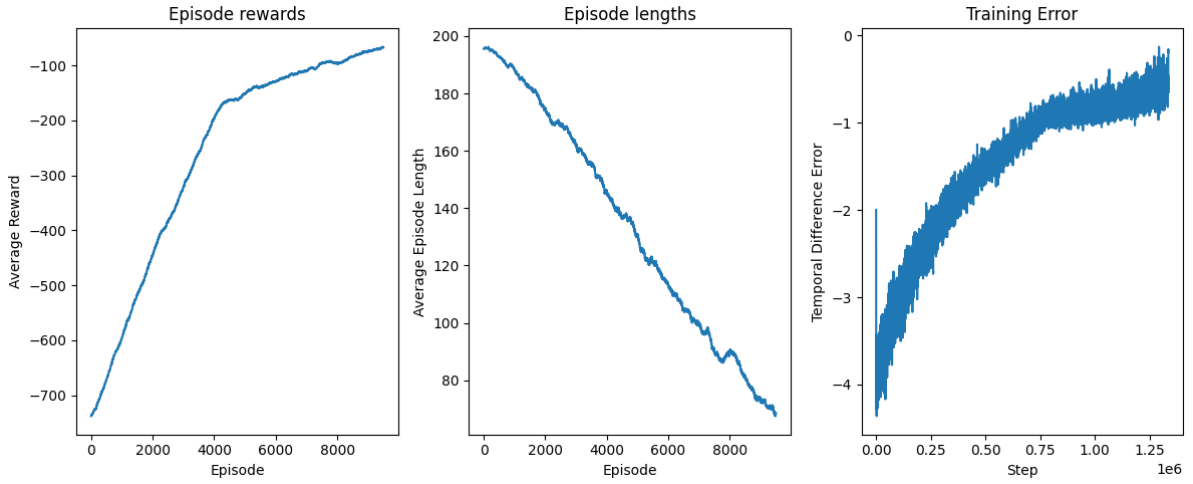


Figure 1: 10k episodes, $\alpha = 0.005$, $\epsilon_{\text{decay}} = \text{start_epsilon} / (\text{n_episodes} / 2)$

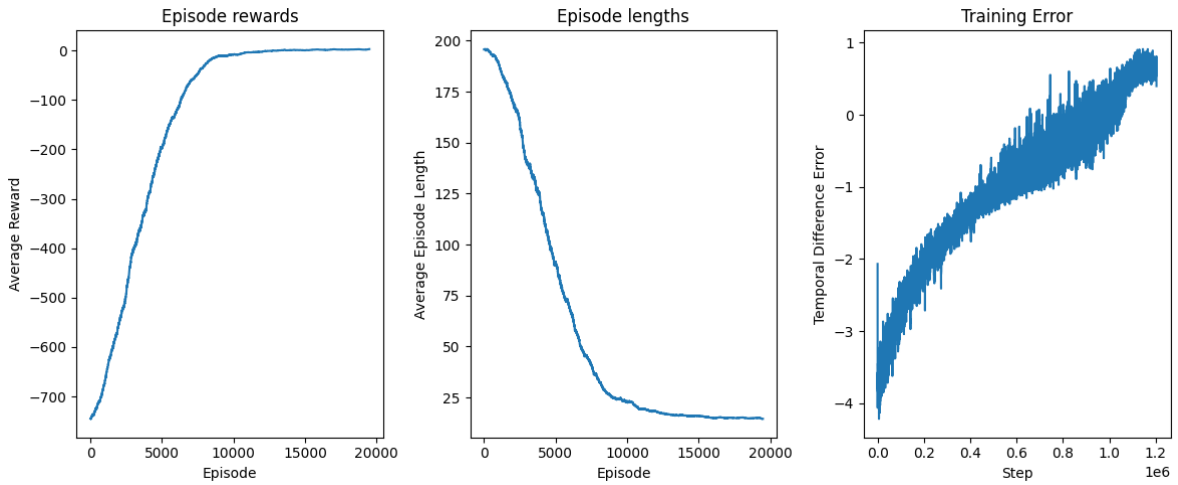


Figure 2: 20k episodes, $\alpha = 0.01$, $\epsilon_{\text{decay}} = \text{start_epsilon} / (\text{n_episodes} / 2)$

Extended training: 50 000–100 000 episodes Beyond 50 000 episodes, all agents with $\alpha = 0.01$ achieved perfect convergence (even with the usage of different gammas), while smaller learning rates caught up more gradually. It's interesting noticing that the usage of slower decay rate with a huge number of episodes didn't obtain better results than the other cases; this validate the faster decay rate used in this deterministic variant.

Table 2: Q-Learning results (Deterministic, extended training).

Episodes	α	γ	Final ϵ	ϵ_{decay}	Win Rate (%)	Avg. Reward $\pm$ Std.
50 000	0.01	0.95	0.1	$\text{start_epsilon} / (\text{n_episodes} / 2)$	100.0	7.88 ± 2.57
50 000	0.005	0.95	0.05	$\text{start_epsilon} / (\text{n_episodes} / 2)$	97.8	3.35 ± 30.95
50 000	0.01	0.9	0.1	$\text{start_epsilon} / (\text{n_episodes} / 2)$	100.0	7.93 ± 2.60
100 000	0.01	0.95	0.1	$\text{start_epsilon} / (\text{n_episodes} / 2)$	100.0	7.93 ± 2.60
100 000	0.01	0.95	0.1	$\text{start_epsilon} / (\text{n_episodes} * 0.8)$	100.0	7.91 ± 2.57

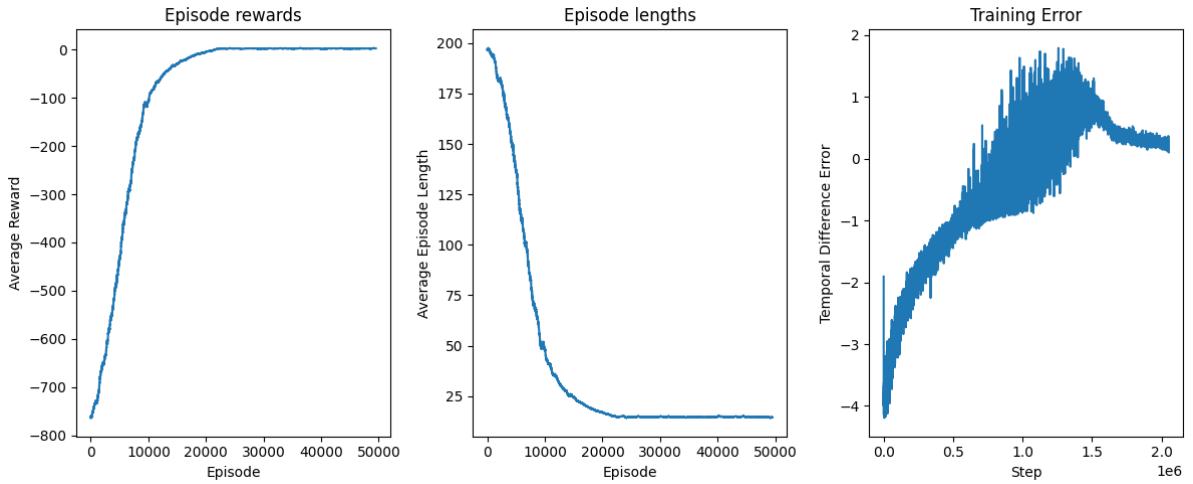


Figure 3: 50k episodes, $\alpha = 0.01$, $\epsilon_{\text{decay}} = \text{start_epsilon} / (\text{n_episodes} / 2)$

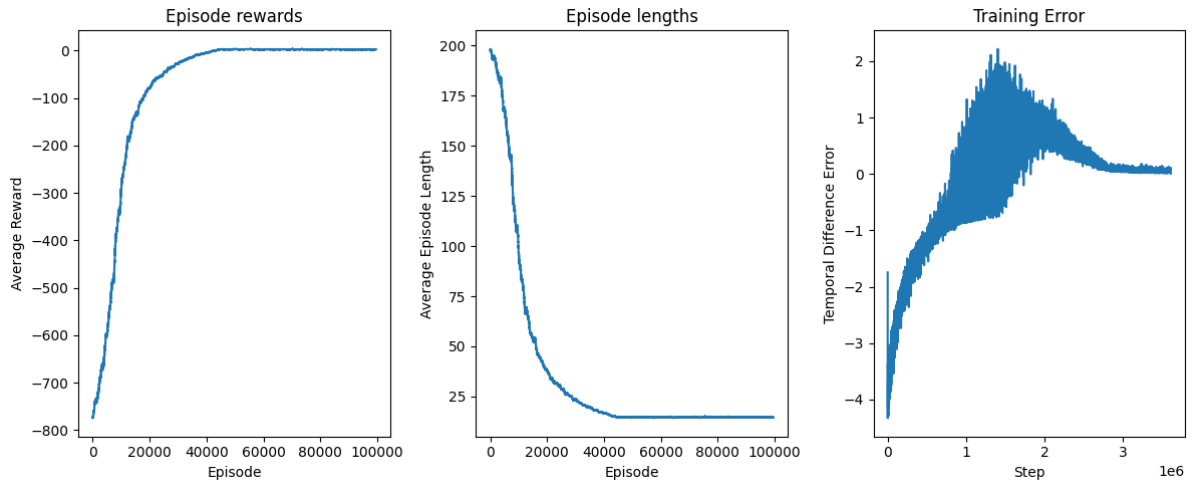


Figure 4: 100k episodes, $\alpha = 0.01$, $\epsilon_{\text{decay}} = \text{start_epsilon} / (\text{n_episodes} / 2)$

Analysis Under deterministic transitions, the Q-table converges to the exact optimal values given sufficient exploration. The learning rate $\alpha = 0.01$ and a moderate decay rate offered the best stability-performance tradeoff, with a win-rate of 7.9.

4.1.3 Stochastic Environment

In stochastic conditions, exploration control (epsilon decay) becomes more critical since the agent must adapt to unpredictable outcomes and occasionally re-explore previously learned states.

Influence of Hyperparameters

- **Learning rate (α):** I started the analysis with small learning rates. $\alpha = 0.01$ again provided the best trade-off, achieving 69.8% win rate after 20k episodes, while $\alpha = 0.005$ lagged at 35.4%. $\alpha = 0.05$ accelerated early learning but caused slight instability before convergence.
- **Epsilon decay (ϵ_{decay}):** In stochastic settings, slower decay is crucial. Too rapid decay may led to premature exploitation of noisy or misleading policies.
- **Episodes:** Longer training didn't provide better returns beyond 50k episodes; convergence plateaued around 83–84%.

Short training: 10 000–20 000 episodes It was clear that 10000 episodes were far from enough; by moving to 20000 win rate started to rise up, also thanks to the usage of a faster learning rate.

Table 3: Q-Learning results (Stochastic, short training).

Episodes	α	γ	Final ϵ	ϵ_{decay}	Win Rate (%)	Avg. Reward $\pm$ Std.
10 000	0.005	0.95	0.1	$\frac{\text{start_epsilon}}{(\text{n_episodes} * 0.8)}$	9.1	-178.87 ± 62.70
20 000	0.005	0.95	0.1	$\frac{\text{start_epsilon}}{(\text{n_episodes} * 0.8)}$	35.4	-120.55 ± 100.77
20 000	0.01	0.95	0.1	$\frac{\text{start_epsilon}}{(\text{n_episodes} * 0.8)}$	69.8	-28.73 ± 75.82

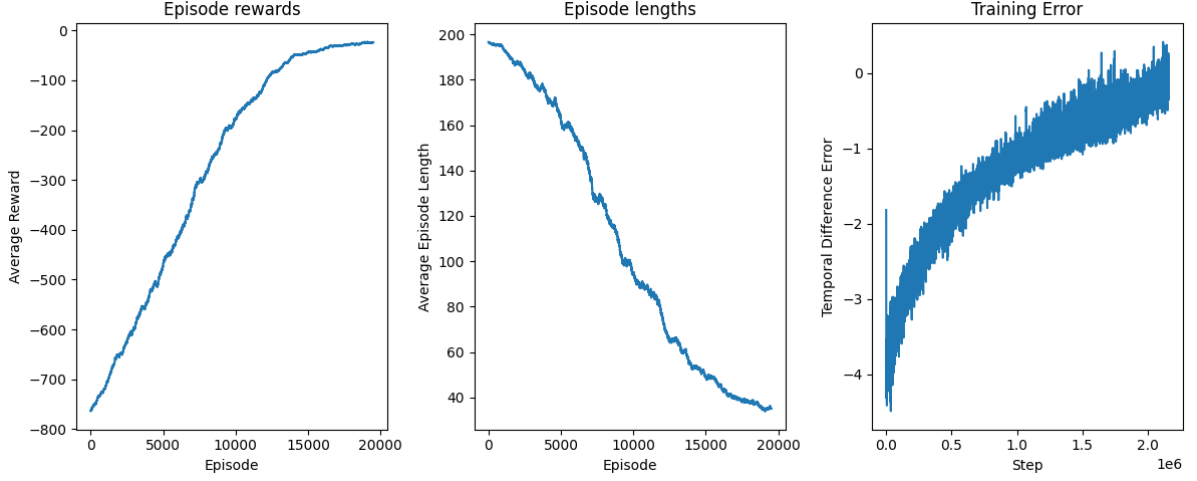


Figure 5: 20k episodes, $\alpha = 0.005$, $\epsilon_{\text{decay}} = 6.25 \times 10^{-5}$

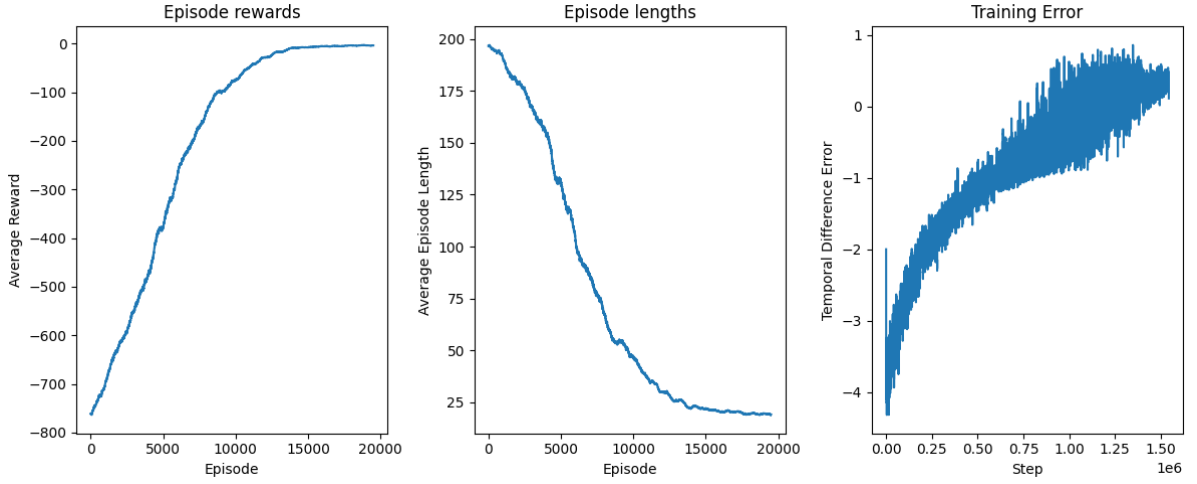


Figure 6: 20k episodes, $\alpha = 0.01$, $\epsilon_{\text{decay}} = 6.25 \times 10^{-5}$

Extended training: 50 000–100 000 episodes At 50k episodes, plots stabilized well at the right plateau, win rates stabilized near 83–84% and the average reward near 5. Even with a faster ϵ_{decay} , we got results comparable to the others, with faster convergence. With a slower learning rate, more than 50000 episodes are needed to get results comparable to faster ones.

Table 4: Q-Learning results (Stochastic, extended training).

Episodes	α	γ	Final ϵ	ϵ_{decay}	Win Rate (%)	Avg. Reward $\pm$ Std.
50 000	0.005	0.95	0.1	$\text{start_epsilon} / (\text{n_episodes} * 0.8)$	80.6	-0.80 ± 34.06
50 000	0.01	0.95	0.05	$\text{start_epsilon} / (\text{n_episodes} * 0.8)$	83.5	5.01 ± 4.77
50 000	0.05	0.95	0.1	$\text{start_epsilon} / (\text{n_episodes} / 2)$	83.6	4.99 ± 4.76
100 000	0.01	0.95	0.1	$\text{start_epsilon} / (\text{n_episodes} * 0.8)$	83.4	5.03 ± 4.71
100 000	0.005	0.95	0.05	$\text{start_epsilon} / (\text{n_episodes} * 0.8)$	82.5	4.90 ± 4.79

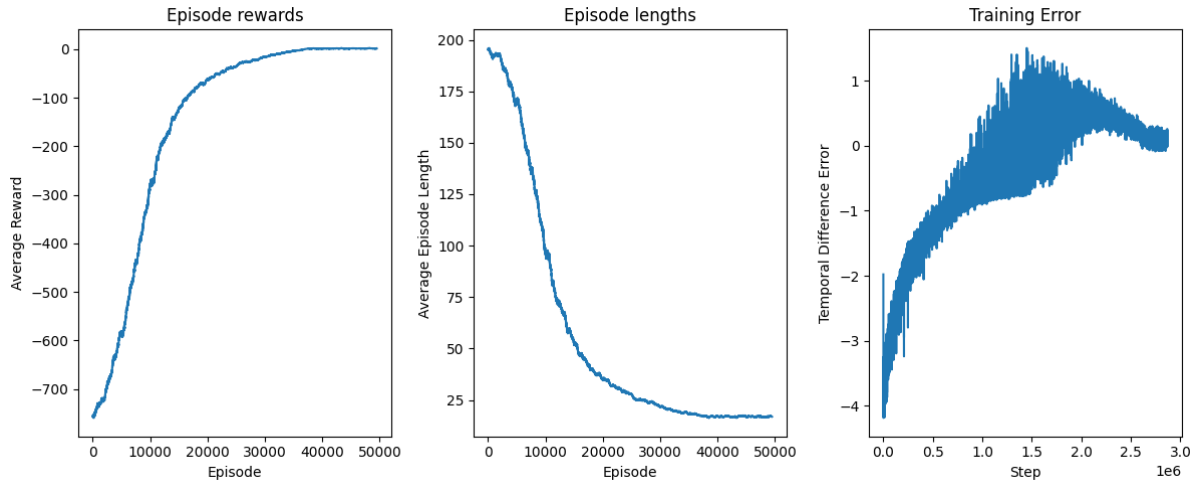


Figure 7: 50k episodes, $\alpha = 0.01$, $\epsilon_{\text{decay}} = \text{start_epsilon} / (\text{n_episodes} * 0.8)$

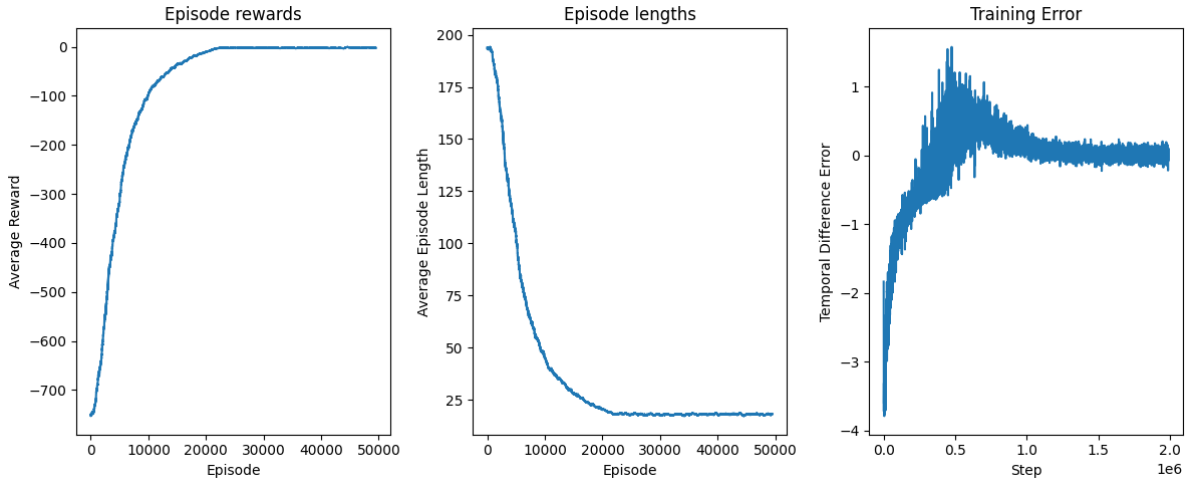


Figure 8: 50k episodes, $\alpha = 0.05$, $\epsilon_{\text{decay}} = \text{start_epsilon} / (\text{n_episodes} / 2)$

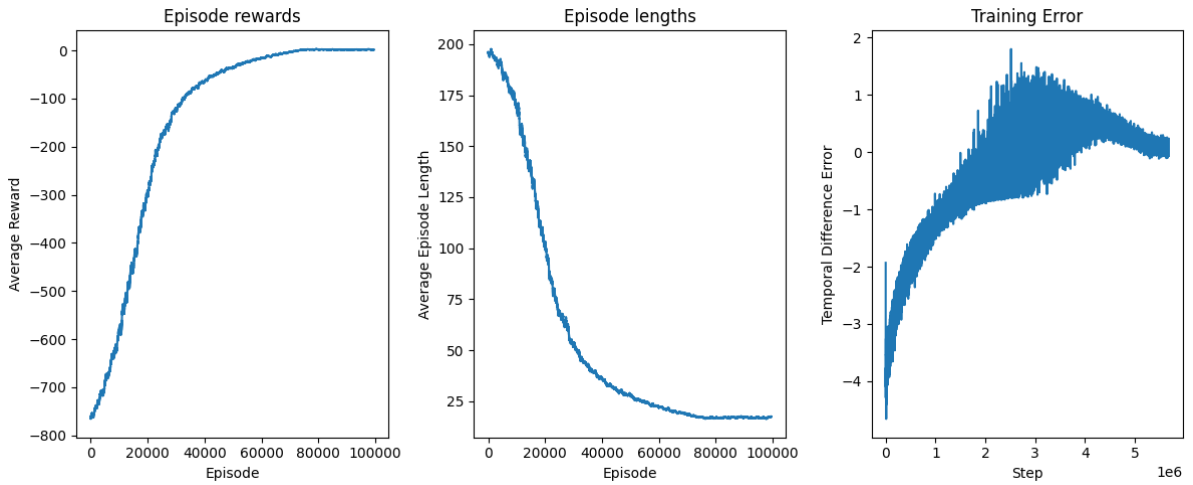


Figure 9: 100k episodes, $\alpha = 0.005$, $\epsilon_{\text{decay}} = \text{start_epsilon} / (\text{n_episodes} * 0.8)$

Analysis In stochastic environments, no combination fully eliminates variability, but hyperparameter tuning can maximize robustness. The best results were achieved with a moderate learning rate ($\alpha = 0.01$), discount factor ($\gamma = 0.95$), and slow decay. These settings yielded steady, efficient policies resistant to random disturbances.

4.2 Deep Q-Network (DQN)

4.2.1 Experimental Setup

The DQN experiments were conducted using the neural architecture and training loop described in Section 3. All runs were performed on CPU using the Gymnasium Taxi-v3 environment, with two variants:

- **Deterministic:** Standard behavior with fixed passenger and weather conditions.
- **Stochastic:** Includes randomness in passenger behavior and weather conditions, increasing the environment’s uncertainty.

The core hyperparameters tuned across experiments include:

- Learning rate $\alpha \in \{0.0001, 0.0005, 0.005\}$
- Epsilon decay rate set to $\epsilon_{\text{decay}} = \text{start_epsilon} / (\text{n_episodes} * 0.8)$
- Final exploration rate fixed to $\epsilon_{\text{final}} = 0.01$
- Discount factor $\gamma = 0.95$
- Episode counts $\in \{1000, 5000, 10000, 15000\}$

Performance was evaluated over 1000 testing episodes with exploration turned off ($\epsilon = 0$), recording the average reward, standard deviation, and win rate (episodes successfully delivering the passenger).

4.2.2 Deterministic Environment

In the deterministic domain, different combinations of learning rate and training duration were tested to evaluate convergence speed and stability.

Short training: 1000–2500 episodes In the initial runs (1000–2500 episodes), the network failed to converge. Rewards remained close to the lower bound (-200), and the win rate was consistently 0%. This indicates that the agent did not experience sufficient exploration and replay iterations to learn meaningful Q-value representations.

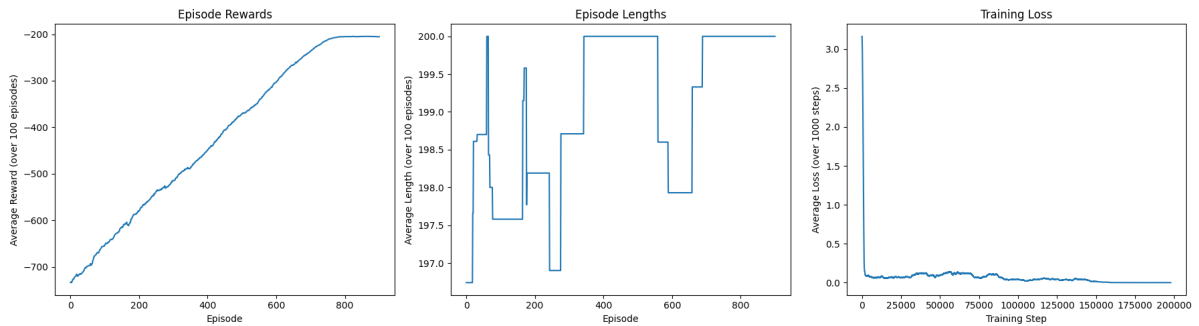


Figure 10: 1000 episodes, $\alpha = 0.0001$, $\epsilon_{\text{decay}} = \text{start_epsilon} / (\text{n_episodes} * 0.8)$

Intermediate training: 5000 episodes When increasing training to 5000 episodes, learning improvements became evident. For example:

- $\alpha = 0.0001 \rightarrow$ Win rate: 100%, Avg. reward: 8.038 ± 2.711
- $\alpha = 0.0005 \rightarrow$ Win rate: 99.1%, Avg. reward: 6.173 ± 19.820

With $\alpha = 0.0001$ the agent reached near-perfect behavior (100% win rate in our tests) and low variance in returns (8.038 ± 2.711). Slightly larger learning rates (e.g., $\alpha = 0.0005$) also solved the task but with higher variance and occasional failures (6.173 ± 19.820), suggesting that a moderate learning rate stabilizes final performance while a slightly larger rate accelerates learning at the cost of transient instability.

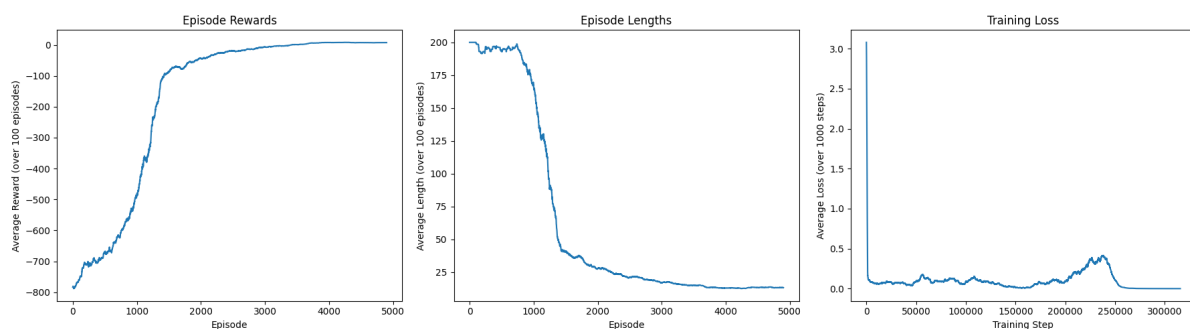


Figure 11: 5000 episodes, $\alpha = 0.0001$, $\epsilon_{\text{decay}} = \text{start_epsilon} / (\text{n_episodes} * 0.8)$

Long training: 10000-15000 episodes Extending training to 10 000-15 000 episodes generally preserved high performance for the deterministic task.

- 10000 episodes, $\alpha = 0.0001 \rightarrow$ Win rate: 100%, Avg. reward: 7.884 ± 2.575
- 15000 episodes, $\alpha = 0.0001 \rightarrow$ Win rate: 100%, Avg. reward: 7.875 ± 2.586

For $\alpha = 0.0001$ the agent reliably achieved 100% win rate with small standard deviation (e.g., 7.884 ± 2.575 at 10 000 episodes and 7.875 ± 2.586 at 15 000). Larger learning rates (e.g., $\alpha = 0.005$) occasionally achieved high win rates but exhibited unstable average rewards and very large variance, which confirms that smaller learning rates are preferable for this environment when using function approximation.

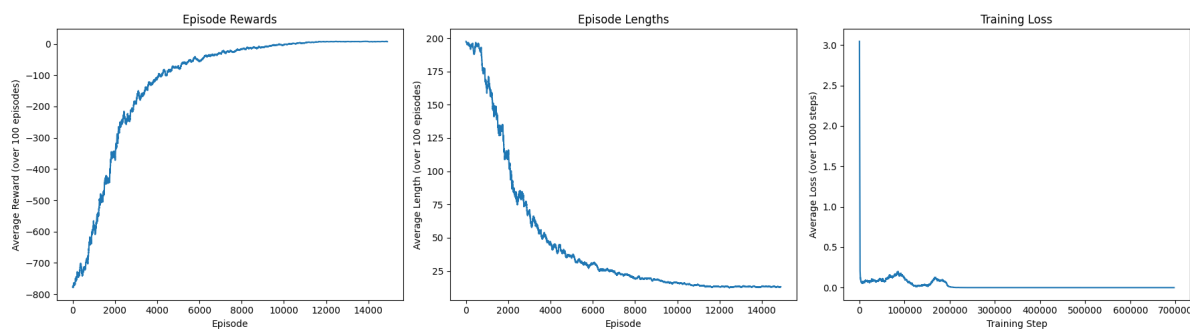


Figure 12: 15000 episodes, $\alpha = 0.0001$, $\epsilon_{\text{decay}} = \text{start_epsilon} / (\text{n_episodes} * 0.8)$

Table 5: Summary of DQN results in deterministic environment.

Episodes	Learning Rate	Final ϵ	Win Rate (%)	Avg. Reward $\pm$ Std.
1000	0.0001	0.01	0.0	-200.0 $\pm$ 0.0
5000	0.0001	0.01	100.0	8.038 $\pm$ 2.711
5000	0.0005	0.01	99.1	6.173 $\pm$ 19.820
10000	0.0001	0.01	100.0	7.884 $\pm$ 2.575
10000	0.0005	0.01	99.7	7.215 $\pm$ 11.642
15000	0.0001	0.01	100.0	7.875 $\pm$ 2.586
15000	0.005	0.01	95.2	-1.864 $\pm$ 44.560

Training loss behaviour and interpretation The recorded training-loss curves follow a characteristic pattern across successful runs:

1. **Initial high loss:** the untrained network starts with large prediction errors, producing a high loss.
2. **Rapid collapse:** after a few gradient steps, the loss sharply drops—often near zero—as the network quickly fits the low-diversity transitions from early episodes.
3. **Transient oscillations:** brief spikes appear when new or rare transitions are encountered, reflecting high TD errors that are rapidly corrected.
4. **Stabilization:** as training progresses, the loss flattens near zero, indicating consistent Q-value predictions and convergence of the policy.

Faster epsilon-decay experiment To evaluate the sensitivity to the exploration schedule, an additional set of deterministic runs was executed with a *faster* epsilon decay. The decay formula remained the same as before, but the parameter `epsilon_decay_factor` was reduced from 0.8 to 0.5. Practically, this makes ϵ reach its final value earlier in training, promoting exploitation sooner.

Observed effects: Learning progress exhibited faster initial improvements, as the average reward rose earlier due to the agent exploiting more quickly the best actions discovered during early exploration. With the standard low learning rate ($\alpha = 0.0001$) and sufficient training duration (5k–10k episodes), final win rates remained high and comparable to the baseline configuration.

Table 6: Deterministic DQN: selected results with faster epsilon decay (factor = 0.5).

Episodes	Learning Rate	Win Rate (%)	Avg. Reward $\pm$ Std.
5 000	0.0001	100.0	7.894 $\pm$ 2.604
10 000	0.0001	100.0	8.044 $\pm$ 2.526

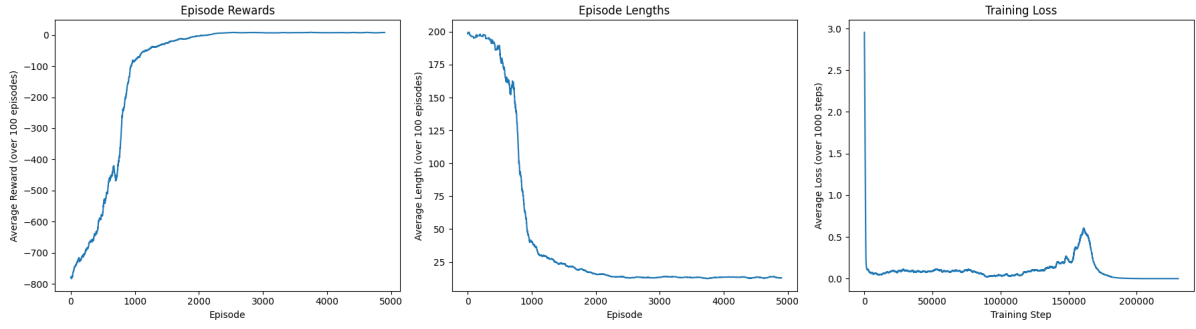


Figure 13: 5000 episodes, $\alpha = 0.0001$, $\epsilon_{\text{decay}} = \text{start_epsilon} / (\text{n_episodes} * 0.5)$

A faster epsilon decay can be beneficial when computational resources are limited and earlier exploitation is desired. However, it also increases sensitivity to early exploration: the agent may prematurely converge to a suboptimal policy if its early experience is insufficiently diverse. To mitigate this risk, one may adopt (i) a slightly smaller learning rate to stabilize weight updates, or (ii) a modest increase in the total number of training episodes to ensure robust convergence.

4.2.3 Stochastic Environment

The stochastic environment introduces environmental variability through random weather and passenger behavior, increasing task difficulty. As expected, performance degraded slightly compared to the deterministic case.

Training duration effects As in the deterministic case, a low number of episodes didn't permit to obtain an avg. reward better than -200 and a win rate better than 0%, so I soon decided to increase it. Training over 10000 to 15000 episodes with $\alpha = 0.0001$ led to good but not perfect performance:

- 10000 episodes, $\alpha = 0.0001 \rightarrow$ Win rate: 82.9%, Avg. reward: 4.801 ± 4.824
- 15000 episodes, $\alpha = 0.0001 \rightarrow$ Win rate: 82.2%, Avg. reward: 4.684 ± 4.732

The results indicate that the agent learned a stable policy but was occasionally misled by the stochasticity.

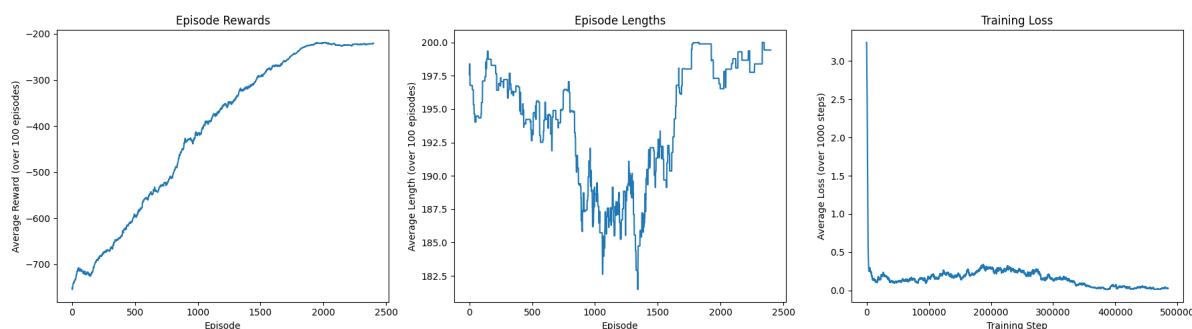


Figure 14: 2500 episodes, $\alpha = 0.0001$, $\epsilon_{\text{decay}} = \text{start_epsilon} / (\text{n_episodes} * 0.8)$

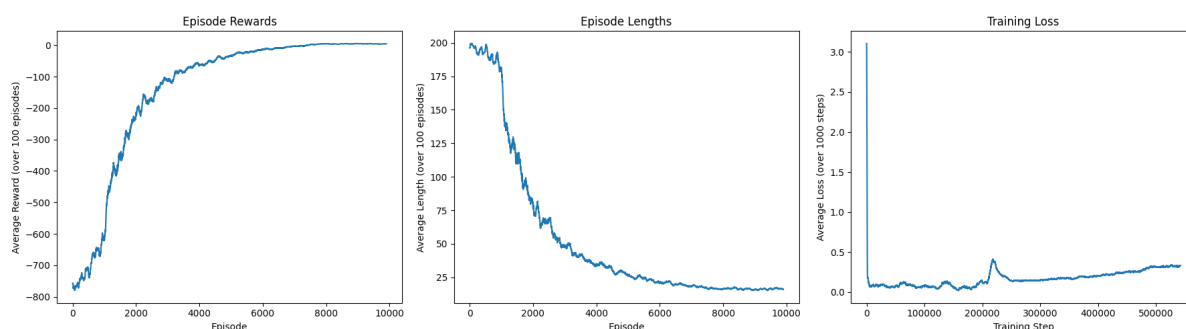


Figure 15: 10000 episodes, $\alpha = 0.0001$, $\epsilon_{\text{decay}} = \text{start_epsilon} / (\text{n_episodes} * 0.8)$

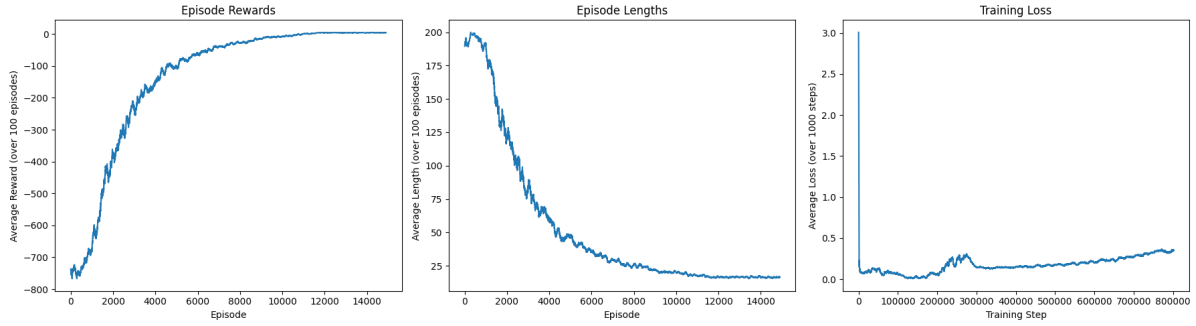


Figure 16: 15000 episodes, $\alpha = 0.0001$, $\epsilon_{\text{decay}} = \text{start_epsilon} / (\text{n_episodes} * 0.8)$

Hyperparameters influence A slightly higher learning rate ($\alpha = 0.0005$) produced slightly worse performance (Win rate: 80.1% , Avg. reward: 4.489 ± 5.177), a trend already seen in the deterministic case, as for increasing the final epsilon (0.05), that led to worse average reward (Win rate: 81.0% , Avg. reward: 3.987 ± 13.791).

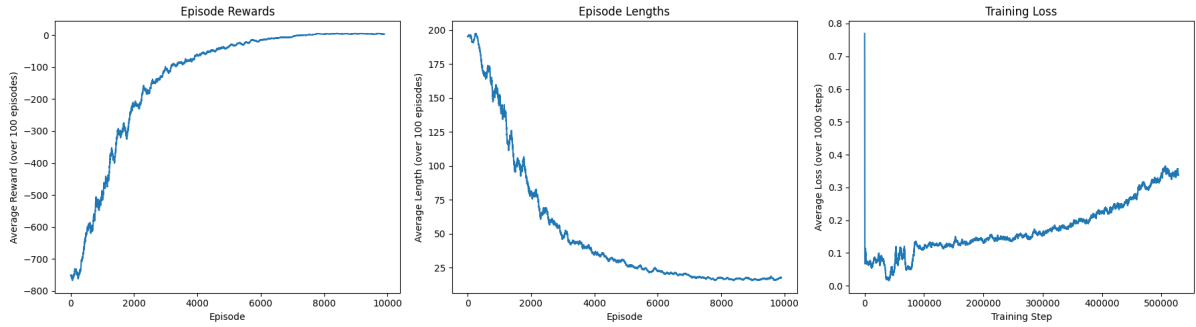


Figure 17: 10000 episodes, $\alpha = 0.0005$, $\epsilon_{\text{decay}} = \text{start_epsilon} / (\text{n_episodes} * 0.8)$

Training loss behaviour Compared to the deterministic case, the training loss in the stochastic environment does not fully stabilize at zero. Instead, it fluctuates around small positive values and often shows a mild increase in the final part of training. This trend reflects the intrinsic randomness of the environment: since transition outcomes and rewards vary, the target Q-values continuously shift, preventing the network from perfectly minimizing the temporal-difference error. In the best configurations (e.g., $\alpha = 0.0001$), this increase remains minor and indicates normal adaptation to noise. Conversely, with higher learning rates (e.g., $\alpha = 0.0005$), the loss grows more markedly, suggesting that large update steps amplify the effect of stochastic variability.

Analysis The DQN successfully generalized to the stochastic environment, albeit with lower average returns. This outcome aligns with expectations, as unpredictable transitions reduce the reliability of learned Q-value estimates. Nevertheless, the convergence trend demonstrates that the agent adapts effectively when sufficient training episodes are provided.

Table 7: Summary of DQN results in stochastic environment.

Episodes	Learning Rate	Final ϵ	Win Rate (%)	Avg. Reward $\pm$ Std.
2500	0.0001	0.05	0.0	-200.0 $\pm$ 0.0
10000	0.0001	0.01	82.9	4.801 $\pm$ 4.824
10000	0.0005	0.01	80.1	4.489 $\pm$ 5.177
10000	0.0001	0.05	81.0	3.987 $\pm$ 13.791
15000	0.0001	0.01	82.2	4.684 $\pm$ 4.732

4.3 Comparative Analysis: Q-Learning vs DQN

Metric	Q-Learning (Det.)	DQN (Det.)	Q-Learning (Stoch.)	DQN (Stoch.)
Win Rate	100%	100%	83.6%	82.9%
Avg. Reward	7.93	8.04	5.03	4.80
Training Time	Low	High	Moderate	High
Convergence	50k eps	5-10k eps	50k eps	10k eps

Table 8: Comparative summary between Q-Learning and DQN.